SIMATS ENGINEERING

CO2_AT3 ERROR ANALYSIS TASK

Student Name: PRADEEP.S

Registration Number: 192521024

Course Code: CSA3711

Course name: SOFTWARE TESTING

1. Online Shopping – Correct ECP and BVA

Errors: The tester has mixed valid and invalid inputs in one equivalence class, even though they should produce different outcomes. The classes are not mutually exclusive and may not cover the entire input domain. Boundary values are omitted, so off-by-one errors can remain undetected. Root causes include misunderstanding that values in one class should be expected to behave similarly and relying on typical values only.

Corrected design (example assumptions: item price must be greater than ₹0, quantity must be an integer from 1 to 100, and discount must be 0–50%):
• Price classes: invalid price ≤ 0; valid price > 0 and within the supported maximum; invalid price above the maximum; non-numeric/blank input.
• Quantity classes: invalid quantity < 1; valid quantity 1–100; invalid quantity > 100; fractional, blank, or non-numeric input.
• Discount classes: invalid discount < 0%; valid discount 0–50%; invalid discount > 50%; malformed input.

BVA examples: quantity −1, 0, 1, 2, 99, 100, 101; discount −1%, 0%, 1%, 49%, 50%, 51%; and price at the minimum supported value and immediately below/above each configured limit. Use the system's actual limits and precision if they differ from these assumptions.

Each test should verify validation messages, calculated total, rounding, and that invalid values are not accepted. ECP reduces redundant testing by selecting representative values from each class; BVA targets the edges where defects are common. Together they provide stronger and more systematic defect detection.

2. Banking Random Testing – Strategy and Coverage Errors

Random testing alone is inadequate for a banking application because random values may rarely land exactly on transaction limits or important combinations. The claim of "high coverage" is unsupported

unless the team defines and measures a coverage criterion. Input diversity is not the same as statement, branch, requirement, or boundary coverage.

Root causes include no requirement-to-test traceability, no defined valid/invalid input classes, insufficient boundary tests, and no systematic check of decision outcomes such as sufficient balance, daily limit, account status, and authorization.

Correct the strategy with ECP and BVA. For an illustrative permitted transfer range of L to U, test below L, L, a valid value just above L, a typical valid value, U just below, U, and above U. Also test zero, negative, malformed amount, insufficient balance, daily limit exceeded, inactive account, and duplicate request. Use decision tables for combinations of balance, account status, transfer amount, and authorization. Use pairwise testing for broader interactions and retain random/fuzz testing as a supplementary technique.

Measure coverage against requirements and decisions, automate regression tests, and verify that failed transactions do not alter balances. Structured techniques make critical boundaries and business rules explicit, improving repeatability and reliability.

3. Incomplete CFG and Path Testing

Errors: the CFG omits decision nodes, false/else edges, loop entry/exit edges, and possibly exception or early-return paths. If the graph does not represent the actual code, independent paths cannot be identified correctly and coverage results are misleading. Root causes may include drawing the graph from an incomplete understanding of the source and counting statements without tracing control transfers.

Correction: build the CFG directly from the source code. Represent each sequential block as a node, each condition as a decision node, and every possible outcome as a directed edge. Include all loop back-edges, loop exits, and error-handling paths. A generic example is Entry → Validate input → [Valid?]; the false edge goes to Error → Exit, while the true edge goes to Process → [Processing successful?]; the true edge goes to Save → Exit, and the false edge goes to Recovery/Error → Exit.

Calculate cyclomatic complexity using $V(G) = E - N + 2P$, where E is edges, N is nodes, and P is connected components. For a single connected module, $P = 1$. Select a basis set of linearly independent paths and design tests to exercise each decision outcome and loop boundary. The exact number and sequence of paths depend on the actual code, which is not supplied in the question.

Review branch coverage and basis-path coverage reports and add tests for zero, one, and multiple loop iterations where applicable. A complete CFG and systematic path testing expose untested logic and improve defect detection.

4. Why 100% Statement Coverage Is Not Enough

Statement coverage means every executable statement ran at least once. It does not ensure that every true and false outcome of a condition was tested. A test may execute the body of an if statement while never testing its else branch. Thus logical defects in comparisons and compound conditions may remain.

For example:

```
if (balance >= amount) {
   approve();
} else {
   reject();
```

}
A test with sufficient balance executes approve(), but without a test for insufficient balance the reject() branch is untested.

Improve the suite with 100% branch coverage where feasible by testing both outcomes of every decision. Add condition coverage for compound Boolean expressions and basis-path testing for independent control-flow paths. For critical logic, consider MC/DC, which demonstrates that each atomic condition can independently affect the decision outcome. Include valid, invalid, boundary, and exception cases.

Construct a CFG from the actual code and map tests to every branch. Statement coverage remains useful, but branch/path/condition coverage provides stronger evidence of logical testing. Coverage metrics must be combined with expected-result assertions and requirements-based testing.

5. Age Validation – Correct Equivalence Classes and BVA

The valid age range is 0–100 inclusive. Defining only "negative" and "positive" classes is incorrect because it treats ages above 100 as valid positive values and fails to distinguish age 0 from valid positive ages. It also does not handle non-numeric, blank, or fractional inputs if those are not allowed.

Correct ECP classes:
1. Invalid: age < 0.
2. Valid: age from 0 through 100 inclusive.
3. Invalid: age > 100.
4. Invalid format: blank, letters, symbols, or non-numeric data.
5. Invalid type if the field requires whole years: fractional age.

BVA tests should include −1, 0, 1, 99, 100, and 101. Also test blank input and values such as "abc" and 2.5 if the application expects an integer. Confirm that 0 and 100 are accepted because the stated range is inclusive, and that −1 and 101 are rejected.

ECP provides representative tests for each distinct expected behavior, while BVA concentrates on the range edges. Together they identify validation errors more effectively than the original two-class design.

6. Condition Coverage – Incomplete Question in Source PDF

The PDF question is cut off after: "A developer claims full Condition Coverage for a complex if(A && B…". The remaining expression and task are not visible, so exact code-specific test cases cannot be responsibly reconstructed from the supplied source.

General analysis: condition coverage requires each atomic Boolean condition to evaluate to both True and False at least once. For a decision such as if (A && B), a minimal illustrative condition-coverage set is:
- Test 1: A = True, B = True → decision True.
- Test 2: A = False, B = True → decision False.
- Test 3: A = True, B = False → decision False.

This set demonstrates both truth values for A and B. Depending on the language, short-circuit evaluation may mean B is not actually evaluated when A is False; test instrumentation or a carefully selected set may be needed to confirm evaluation.

Condition coverage is not always the same as branch coverage or MC/DC. MC/DC requires showing that each condition independently changes the overall decision outcome while other conditions are held

fixed. Please check the original full question for its exact requirements and any additional Boolean expression.

7. Discount System – Decision Table Testing

The decision table is incomplete if it omits combinations of Premium Member and Coupon Applied, assumes one condition is always false, or fails to define the expected action for every meaningful rule. These omissions can hide incorrect stacking, eligibility, and price-calculation behavior.

Assume a coupon is valid and discount rules are specified as follows: premium members receive a member discount; an applied coupon gives a coupon discount; stacking is allowed only if the business policy says so. The question does not provide actual percentages or stacking rules, so these must be confirmed rather than invented.

Illustrative decision table:
Rule 1: Premium = No, Coupon = No → no discount.
Rule 2: Premium = No, Coupon = Yes → coupon discount, if valid.
Rule 3: Premium = Yes, Coupon = No → premium discount.
Rule 4: Premium = Yes, Coupon = Yes → apply both only if stacking is allowed; otherwise apply the policy-defined priority or reject the coupon with a clear message.

Add conditions for coupon validity/expiry, minimum purchase, product eligibility, and whether discounts may exceed the subtotal. Test the four core combinations, plus invalid/expired coupon and discount-boundary cases. Verify final price, rounding, displayed messages, and that the total never becomes negative if prohibited by the specification.

A complete decision table enumerates meaningful condition combinations and maps each to an expected action. It prevents omitted rules and gives a traceable basis for regression tests.

8. Loop Testing – Missing Zero and Maximum-Plus-One Cases

Testing only n = 1 and n = max misses important loop behavior. At n = 0, the loop may not execute, so initialization and skip logic need checking. At n = max + 1, the system should reject the input or handle it according to the specification. Without these cases, off-by-one errors, incorrect termination, and buffer or resource-limit defects may remain.

For a loop with permitted iteration count 1 through max, test 0, 1, 2, a typical middle value, max−1, max, and max+1. If zero is valid, verify correct empty-result behavior; if it is invalid, verify safe rejection. Test loop termination, correct number of iterations, and output values.

For nested loops, test the inner loop with 0, 1, and multiple iterations while the outer loop also takes 0, 1, and multiple iterations where allowed. Include combinations such as an empty inner collection, maximum inner size, and boundary sizes in both loops. Avoid attempting every combination when the space is large; prioritize boundaries and interaction cases.

Use CFG review, branch coverage, loop-boundary testing, and timeouts for detecting non-termination. These cases help ensure path integrity and reliable loop exits.

9. Safety-Critical Braking – Error Guessing, BVA and State Transitions

Relying only on Error Guessing is inadequate for a safety-critical braking system because ad-hoc tests depend on the tester's experience and may omit important thresholds, state sequences, and requirements. It provides no systematic assurance that all critical conditions or transitions have been tested.

Retain error guessing as a supplementary technique, targeting likely faults such as sensor disconnection, stale readings, invalid values, delayed signals, and conflicting inputs. Add BVA around specified braking thresholds: test just below, exactly at, and just above each threshold, using the actual units and sensor precision from the requirements. Include invalid sensor ranges and missing data.

Use State Transition Testing to model states such as Normal Driving, Hazard Detected, Braking, ABS/Control Active (if present), Stopped, and Sensor Fault. The real state model must follow the vehicle's specification. Test all legal transitions and critical illegal transitions, including rapid changes in sensor readings, hazard detection during another state, and recovery from sensor faults. Verify expected braking command, warnings, timing, and safe fallback behavior.

Add requirements traceability, independent review, simulation or hardware-in-the-loop tests, and regression testing. BVA targets threshold errors; state-transition tests target sequence and state-dependent faults. Together they provide stronger safety evidence than ad-hoc testing alone, though they do not replace formal safety analysis and applicable standards.

10. CFG Missing Else Branches – Corrected Representation

A CFG that omits else edges is incomplete because every decision can have multiple outcomes. If false edges and error-handling nodes are missing, tests may execute all statements on the main path and still achieve 0% branch coverage for error paths. The graph and its coverage report therefore do not represent the actual program behavior.

Use a corrected conceptual CFG:
Entry → Decision 1: Input valid?
• True → Decision 2: User authorized?
• True → Process request → Decision 3: Operation successful?
• True → Commit result → Exit.
• False → Roll back / report error → Exit.
• False → Authorization error → Exit.
• False → Validation error → Exit.

This representation explicitly includes the false/else edges. It is illustrative; replace it with the actual conditions in the source module.

Independent paths should include: (1) invalid input → validation error; (2) valid input but unauthorized → authorization error; (3) valid and authorized with successful operation → commit; and (4) valid and authorized with operation failure → rollback/error. Additional paths are required for each actual decision and loop in the source.

Calculate cyclomatic complexity from the real CFG and create a basis set of independent paths. Run tests for both outcomes of every decision and confirm error messages, rollback behavior, and state consistency. Branch coverage is essential here; path testing provides additional control-flow confidence. The corrected graph helps prevent error-handling logic from remaining untested.

Key Concepts – Quick Revision

- **ECP:** Divides inputs into valid and invalid classes with similar expected behavior.
- **BVA:** Tests values at and immediately around range boundaries.
- **Statement coverage:** Checks whether each executable statement ran.
- **Branch coverage:** Checks both outcomes of each decision.
- **Condition coverage:** Checks that each atomic Boolean condition is True and False.
- **MC/DC:** Checks that each condition independently affects the overall decision.
- **CFG:** A graph of statements/blocks and possible control transfers.
- **Cyclomatic complexity:** $V(G) = E - N + 2P$; for one connected component, $E - N + 2$.
- **State transition testing:** Tests state changes caused by events, including valid and invalid transitions.

Before Submission

- Fill in your name and registration number.
- Question 6 is incomplete in the supplied PDF; obtain the full question from the lecturer or original document.
- Confirm example business rules and limits against the actual assignment requirements.
- Read and understand the answers and complete the academic-integrity declaration yourself.